Numpy

September 2, 2023

1 NumPy

- Numeric python

2 Why NumPy

- It's Fast
- Behind the scenes optimization written in C
- Vectorization via broadcasting (avoiding loops)
- Backbone of other python scientific packages

3 What will we see

- Most useful functions
- NumPy datatypes and attributes
- Creating arrays
- Viewing arrays and matrices
- Manipulating and comparing arrays
- Sorting arrays
- Use cases

3.1 Links :

- Numpy Docs

[2]:
```
import numpy as np
```

3.2 DataTypes and Attributes

- NumPy's main datatype is ndarray (n-dimesional array)

[3]:
```
a1 = np.array([1,2,3])
a1
```

[3]:
```
array([1, 2, 3])
```

[4]:
```
type(a1)
```

[4]:
```
numpy.ndarray
```

[5]:
```
a2 = np.array([[2, 3, 3, 3, 1], [2, 1, 3, 4, 4]])
a2
```

[5]:
```
array([[2, 3, 3, 3, 1],
       [2, 1, 3, 4, 4]])
```

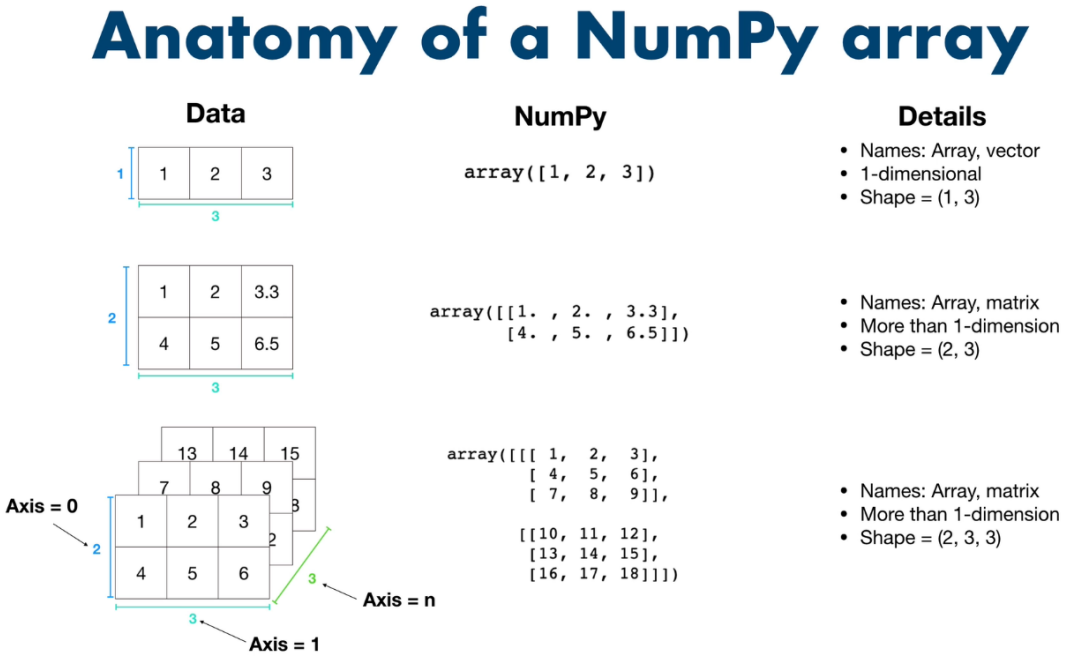
[6]:
```
a3 = np.array([[[1, 3, 1, 5, 3], [4, 3, 1, 1, 2], [3, 5, 1, 4, 2]],
 [[4, 2, 3, 5, 1], [5, 3, 3, 5, 2], [1, 1, 4, 4, 1]],
 [[4, 3, 1, 1, 4], [4, 5, 1, 4, 4], [4, 2, 5, 2, 4]]])
a3
```

[6]:
```
array([[[1, 3, 1, 5, 3],
        [4, 3, 1, 1, 2],
        [3, 5, 1, 4, 2]],

       [[4, 2, 3, 5, 1],
        [5, 3, 3, 5, 2],
        [1, 1, 4, 4, 1]],

       [[4, 3, 1, 1, 4],
        [4, 5, 1, 4, 4],
        [4, 2, 5, 2, 4]]])
```

4 Understanding ndarray



[7]:
```
a1.shape, a2.shape, a3.shape
```

[7]:
```
((3,), (2, 5), (3, 3, 5))
```

[8]:
```
a1.ndim, a2.ndim, a3.ndim
```

[8]:
```
(1, 2, 3)
```

[9]:
```
a1.size, a2.size, a3.size
```

[9]:
```
(3, 10, 45)
```

[10]:
```
type(a1), type(a2), type(a3)
```

[10]:
```
(numpy.ndarray, numpy.ndarray, numpy.ndarray)
```

[11]:
```
# Create a DataFrame from NumPy array
import pandas as pd
df = pd.DataFrame(a2)
df.head()
```

[11]:

	0	1	2	3	4
0	2	3	3	3	1
1	2	1	3	4	4

[12]:
```
## Create standard matrix
np.ones((5,5))
```

[12]:
```
array([[1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.]])
```

[14]:
```
np.zeros((5,5))
```

[14]:
```
array([[0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0.]])
```

[18]:
```
np.arange(-5, 5, 1)
```

[18]:
```
array([-5, -4, -3, -2, -1,  0,  1,  2,  3,  4])
```

[19]:
```
np.random.randint(0, 10, size=(5,5))
```

[19]:
```
array([[3, 4, 9, 0, 3],
       [7, 1, 9, 2, 4],
       [3, 2, 6, 0, 4],
       [3, 0, 7, 8, 9],
       [1, 9, 1, 5, 8]])
```

[20]:
```
np.random.random((5,5))
```

[20]:
```
array([[0.26951886, 0.86607313, 0.94865102, 0.27618244, 0.2266554 ],
       [0.28588982, 0.18929117, 0.42067078, 0.4809572 , 0.83214684],
       [0.61107028, 0.22514334, 0.79704994, 0.32273256, 0.42246445],
       [0.60140158, 0.38143608, 0.56836018, 0.12404502, 0.32147452],
       [0.50341565, 0.34030075, 0.28846833, 0.2155421 , 0.80257531]])
```

[22]:
```
np.random.rand(5,5)
```

[22]:
```
array([[0.18373496, 0.77147216, 0.37621757, 0.49251158, 0.15250713],
       [0.96636541, 0.66131322, 0.36573068, 0.40520883, 0.09016344],
       [0.79580339, 0.72377837, 0.32524611, 0.00909022, 0.70454124],
       [0.06967594, 0.64671331, 0.5629788 , 0.21278936, 0.37463556],
       [0.9496818 , 0.56735702, 0.97530569, 0.75732084, 0.6876534 ]])
```

[42]:
```
# Pseudo-random Number
np.random.seed(2)
np.random.rand(2)
```

[42]:
```
array([0.4359949 , 0.02592623])
```

[47]:
```
np.random.rand(2)
```

[47]:
```
array([0.62113383, 0.52914209])
```

4.1 Array manipulation

[51]:
```
arr = np.random.randint(10, size=(5,5))
```

[52]:
```
np.unique(arr)
```

[52]:
```
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

[53]:
```
arr
```

[53]:
```
array([[8, 2, 9, 6, 5],
       [6, 6, 6, 3, 8],
       [2, 1, 4, 8, 1],
       [6, 9, 5, 1, 2],
       [4, 7, 6, 4, 5]])
```

[58]:
```
# arr[dim1, dim2]
arr[:2,:2]
```

[58]:
```
array([[8, 2],
       [6, 6]])
```

4.2 Manipulating and Comparing arrays

[61]:
```
arr = np.random.randint(10, size=(3,3))
arr
```

[61]:
```
array([[0, 8, 5],
       [4, 2, 9],
       [7, 1, 9]])
```

[62]:
```
ones = np.ones((3,3))
```

[63]:
```
ones
```

[63]:
```
array([[1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.]])
```

[64]:
```
arr - ones
```

[64]:
```
array([[-1.,  7.,  4.],
       [ 3.,  1.,  8.],
       [ 6.,  0.,  8.]])
```

[65]:
```
arr * arr
```

[65]:
```
array([[ 0, 64, 25],
       [16,  4, 81],
       [49,  1, 81]])
```

[67]:
```
arr2 = np.random.randint(10, size=(5,5))
arr2
```

[67]:
```
array([[1, 5, 9, 2, 0],
       [0, 4, 6, 3, 1],
       [8, 5, 9, 5, 4],
       [2, 7, 8, 7, 3],
       [4, 8, 6, 3, 8]])
```

4.3 BroadCasting

[73]:
```
a1 = np.random.randint(10, size=(1,5))
a1
```

[73]:
```
array([[7, 5, 9, 4, 5]])
```

[75]:
```
a2 = np.random.randint(10, size=(2,5))
a2
```

[75]:
```
array([[2, 4, 0, 6, 6],
       [8, 6, 9, 5, 1]])
```

[77]:
```
a2 * a1
```

[77]:
```
array([[14, 20,  0, 24, 30],
       [56, 30, 81, 20,  5]])
```

4.4 Aggregation

[83]:
```
large_arr = np.random.random(100)
```

[85]:
```
%timeit sum(large_arr)
%timeit np.sum(large_arr)
```

```
104 µs ± 1.2 µs per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
6.3 µs ± 110 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops each)
```

[87]:
```
np.max(large_arr)
```

[87]:
```
0.9994736465490929
```

[88]:
```
np.min(large_arr)
```

[88]:
```
0.0006491343871583011
```

[89]:
```
np.std(large_arr)
```

[89]:
```
0.29803140594071653
```

[91]:
```
# Variance = measure of average deviation between data points in the input to␣
 ↪its mean
# High variance = wider range of numbers
# Lower variance = lower range of numbers
np.var(large_arr)
```

[91]:
```
0.08882271892700015
```

[92]:
```
np.sqrt(np.var(large_arr))
```

[92]:
```
0.29803140594071653
```

[93]:
```
np.std(large_arr)
```

[93]:
```
0.29803140594071653
```

4.5 Reshape and Dot Product

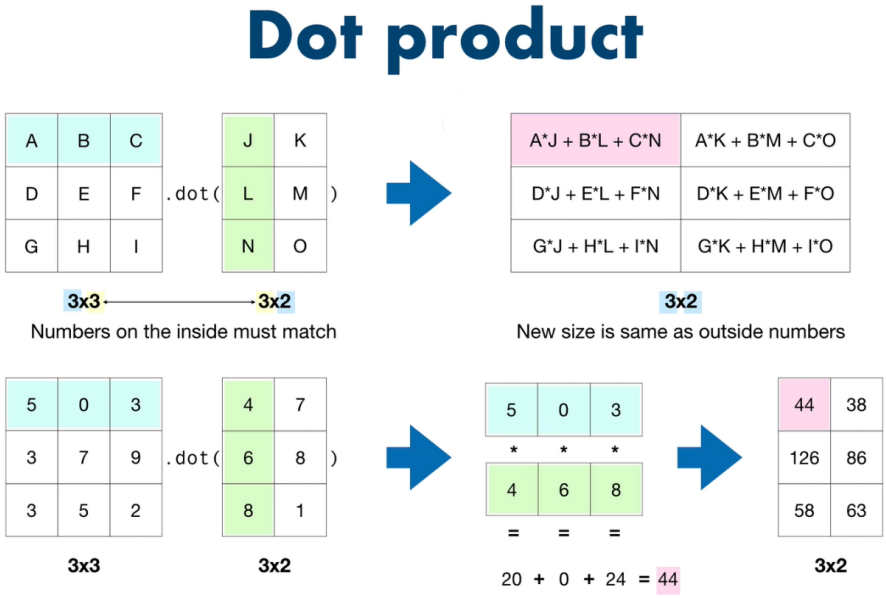
[96]:
```
arr = np.random.random((5,5))
arr
```

[96]:
```
array([[0.46374731, 0.08064275, 0.90738015, 0.59116493, 0.38596417],
       [0.91031662, 0.09205067, 0.33224385, 0.64508162, 0.18359928],
       [0.37370312, 0.01078191, 0.67872426, 0.08433849, 0.59170581],
       [0.81236668, 0.76500616, 0.01183264, 0.27061555, 0.66639865],
       [0.70499555, 0.70518235, 0.72359803, 0.11944108, 0.61049873]])
```

[98]:
```
arr.reshape(25,1)
```

[98]:
```
array([[0.46374731],
       [0.08064275],
       [0.90738015],
       [0.59116493],
       [0.38596417],
       [0.91031662],
       [0.09205067],
       [0.33224385],
       [0.64508162],
       [0.18359928],
       [0.37370312],
       [0.01078191],
       [0.67872426],
       [0.08433849],
       [0.59170581],
       [0.81236668],
       [0.76500616],
       [0.01183264],
       [0.27061555],
       [0.66639865],
       [0.70499555],
       [0.70518235],
       [0.72359803],
       [0.11944108],
       [0.61049873]])
```

5 Dot Product



[109]:
```
arr1 = np.random.randint(0, 10, size=(3,2))
arr2 = np.random.randint(0, 10, size=(3,2))
```

[110]:
```
arr1
```

[110]:
```
array([[7, 7],
       [1, 2],
       [5, 4]])
```

[111]:
```
arr2
```

[111]:
```
array([[7, 7],
       [0, 7],
       [4, 5]])
```

[112]:
```
np.dot(arr1, arr2)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[112], line 1
----> 1 np.dot(arr1, arr2)

File <__array_function__ internals>:200, in dot(*args, **kwargs)

ValueError: shapes (3,2) and (3,2) not aligned: 2 (dim 1) != 3 (dim 0)
```

[113]:
```
np.dot(arr1, arr2.T)
```

[113]:
```
array([[98, 49, 63],
       [21, 14, 14],
       [63, 28, 40]])
```

[115]:
```
arr1 @ arr2.T
```

[115]:
```
array([[98, 49, 63],
       [21, 14, 14],
       [63, 28, 40]])
```

[ ]:
```

```